TEACHING GUIDE · MBOT2 ROVER ROBOTICS PROGRAM

Grade 7 Foundations

Think Like a Robot

20	1 h	mBot2	Block-based coding
SESSIONS	EACH	WHEELED BASE	CODING MODE

★ Final project: Rescue Run

An autonomous mission: follow the line, avoid a surprise obstacle, stop precisely in the rescue zone, celebrate. Two scored attempts + a short team presentation.

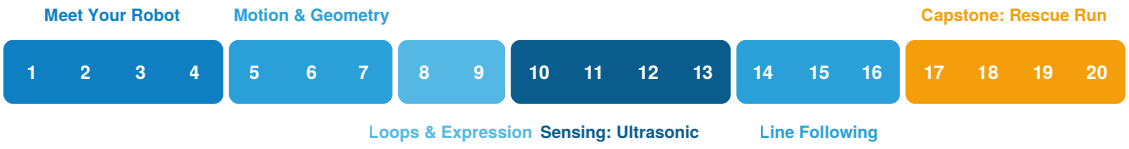
Tutor edition — each session is one page: concept to teach · timed lesson flow · code · pitfalls · expected result with success criteria. Software: mBlock 5 · Makeblock App · 1 kit per 2–3 students.

Grade 7 at a Glance

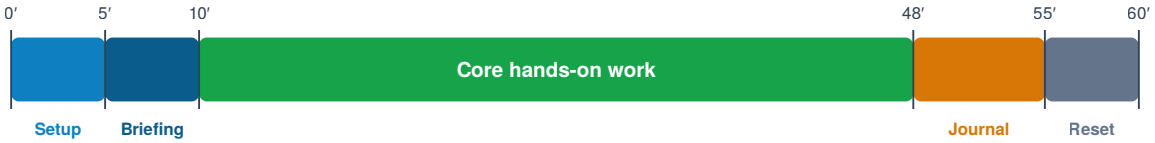
Learning objectives — by the end of the year, students can:

- Assemble and maintain the mBot2 wheeled robot
- Explain the sense–decide–act loop and each sensor's role
- Write block programs: sequences, loops, conditionals, variables, events
- Calibrate precise motion; debug with predict → test → observe → fix
- Complete an autonomous line + obstacle mission (Rescue Run)

The 20 sessions, colored by unit ✱ in session titles = buffer session (tolerates compression if the schedule slips)



Every session runs on the same rhythm



Non-negotiables for 1-hour sessions: robots stay assembled between sessions on a labeled cart shelf · code saved to mBlock cloud accounts every session · battery rotation with a per-team battery manager · extension cards at stations so fast teams never wait.

Known friction points this year: first firmware pairing is slow (teacher dry run before Session 4) · the Shield's battery is built in — charge robots via USB-C before class · sensors must be mounted straight · recalibrate the line sensor whenever floor or light changes · the Rover box stays sealed all year.

The Boxes — Official Contents Reference

Grade 7 uses only Box 1. The Rover add-on box stays sealed until Grade 8 — store it safely.

BOX 1 · “mBot2”

- **CyberPi** ×1
- **mBot2 Shield** ×1 · battery built in
- **Chassis** ×1
- **Encoder motor** ×2
- **Wheel hub** ×2
- **Slick tyre** ×2
- **Mini wheel** ×1
- **Ultrasonic Sensor 2** ×1
- **Quad RGB Sensor** ×1
- **Motor cable** ×2
- **mBuild cable 10 cm** ×2
- **mBuild cable 20 cm** ×1
- **M4×25 mm screw** ×6
- **M4×14 mm screw** ×6
- **M4×8 mm screw** ×6
- **M2.5×12 mm screw** ×4
- **Line-following track map** ×1
- **USB cable** ×1
- **Screwdriver** ×1

Quantities from the official packing list. Bluetooth dongle NOT included — Makeblock advises it for older computers (sold separately).

SESSION 1 / 20 · MEET YOUR ROBOT

What Is a Robot?

Goal of this hour: Students can explain the sense–decide–act loop and apply it to everyday machines.

YOU NEED — PIECES & MATERIALS

- **mBot2 box** — full inventory vs. the box's parts chart — mBot2 box
- **Printed cards** — robot-or-not deck — Classroom
- **Engineering journal** — 1 per student — Classroom
- **Sorting trays** — sort screws by size now — Classroom

TEACH IT — THE CONCEPT

A robot is a machine that **senses** its environment, **decides** what to do using a program, and **acts** on the world — then repeats that loop, often many times per second. A hammer acts but never senses; a thermometer senses but never acts. A robot needs all three.

Run the card sort: give pairs 10–12 cards with machines (vacuum robot, drone, calculator, thermostat, hammer, automatic door, washing machine…). Teams sort into "robot" / "not a robot" and must justify each choice using the three-part test. Disagreements are gold — a thermostat is a great edge case.

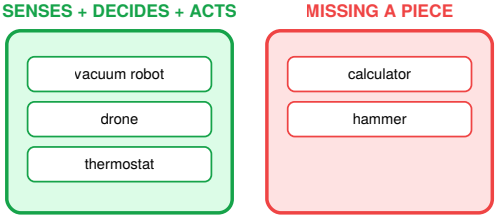
Watch out

- Only the mBot2 box opens this year — the sealed Rover box is Grade 8's reveal; store it safely.
- Photograph the sorted kit; it becomes the reference when parts go missing.

LESSON FLOW (45′ CORE)

8′	Card sort in pairs; teams defend two of their choices to the class.
7′	Board the sense–decide–act loop; test it against the disputed cards.
12′	Form teams of 2–3; assign rotating roles: driver, navigator, documenter.
13′	Inventory the mBot2 box against its printed parts chart; peel the CyberPi screen film. The Rover box stays sealed until Grade 8.
5′	Journal setup + first entry: three robots you met this week without noticing.

◉ EXPECTED RESULT AT THE END OF THIS SESSION



- ☑ Every student can state the three-part robot test
- ☑ Kit inventory checked and signed off
- ☑ Teams formed, roles written in the journal

SESSION 2 / 20 · MEET YOUR ROBOT

Assembly I

Goal of this hour: Chassis, motors and wheels assembled correctly, following the visual guide.

YOU NEED — PIECES & MATERIALS

Chassis
mBot2 box

Encoder motor ×2
check orientation!
mBot2 box

Wheel hub + slick tyre
×2 each: hub + tyre
mBot2 box

Motor cable ×2
×2 — plug in first
mBot2 box

M4 screws
M4×8 ×4 + M2.5×12 ×2
mBot2 box

Screwdriver
in the box
mBot2 box

TEACH IT — THE CONCEPT

The mBot2 build guide is visual (almost no text) — that is deliberate: reading an exploded diagram is an engineering skill. Teach students to check each diagram for three things before touching a part: **which parts**, **which orientation**, **which screws**.

The single most common error is mounting a motor backwards or swapping left/right motor cables. Institute a rule: before wheels go on, the teacher (or a checker team) verifies motor orientation against the guide.

LESSON FLOW (45′ CORE)

5′	Demo: how to read one exploded diagram (parts → orientation → screws). Use guide step ② as the example.
30′	Teams build guide steps ①–③: motor cables in, wheels pressed together, motors into chassis (M4×8), wheels onto shafts (M2.5×12). Screws finger-tight first, then snug.
5′	Teacher checkpoint: motor orientation + cable routing before wheels go on.
5′	Journal: one diagram that was confusing and how you decoded it.

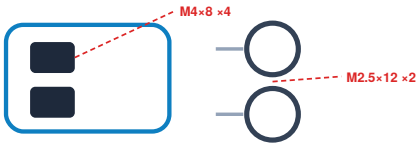
QUICK START GUIDE — THIS SESSION

Steps	① – ③ (printed pp. 9–10)
Screws	M4×8 ×4 (motors) · M2.5×12 ×2 (wheels)
Sequence	cables into motors FIRST · hub pressed into tyre · motors into chassis · wheels on

Watch out

- Plug each motor cable in BEFORE the motor goes into the chassis — the port is hard to reach after (guide step ① order).
- The four long M4×25 screws are Session 3's (shield) — if a team reaches for them today, they're on the wrong step.
- Do not overtighten into plastic; snug is enough.
- Half-built is fine — the cart shelf keeps work-in-progress safe.

◉ EXPECTED RESULT AT THE END OF THIS SESSION



motors into chassis · wheels onto shafts

redrawn from guide steps ①–③ (pp. 9–10)

- ☑ Chassis + motors mounted, orientation verified
- ☑ No leftover "mystery screws"
- ☑ Work-in-progress stored safely on the team shelf

SESSION 3 / 20 · MEET YOUR ROBOT

Assembly II

Goal of this hour: Robot fully assembled, cables managed, battery installed — and QC-checked by another team.

YOU NEED — PIECES & MATERIALS

- **CyberPi** — the brain — mBot2 box
- **mBot2 Shield** — battery is built in — charged? — mBot2 box
- **Ultrasonic Sensor 2** — front, facing exactly forward — mBot2 box
- **Quad RGB Sensor** — under the nose — mBot2 box
- **Mini wheel** — mounts with the RGB sensor — mBot2 box
- **mBuild cables** — 10 cm ×2 — NOT the 20 cm — mBot2 box
- **M2.5×12 screws** — M4×14 ×4 + M4×25 ×4 — mBot2 box
- **Screwdriver** — mBot2 box

TEACH IT — THE CONCEPT

Finish the build: the mBot2 Shield (its battery is built in — charge it beforehand), CyberPi on top, sensor connections (ultrasonic front, line-follower under the nose). Cable management is not cosmetic — a loose cable in a wheel is the #1 cause of "my robot randomly stops".

Introduce peer quality control: each team inspects another team's robot against a four-point checklist (wheels turn freely, cables tucked, battery secured, nothing rattles when gently shaken). Signing another team's checklist creates accountability and sharp eyes.

LESSON FLOW (45′ CORE)

25′	Finish guide steps ④–⑨: Quad RGB + mini wheel underneath (M4×14), ultrasonic up front (M4×14), shield on top (M4×25 — the four long ones), motors to EM1/EM2, CyberPi slides in last; route and tuck everything.
10′	Cross-team QC: inspect + sign another team's checklist; fix anything flagged.
5′	Shake test + wheel spin test in front of the class.
5′	Journal: what did QC catch on the robot you inspected?

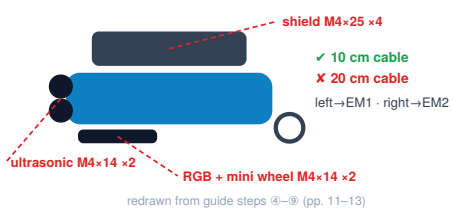
QUICK START GUIDE — THIS SESSION

Steps	④ – ⑩ (printed pp. 11–14)
Screws	M4×14 ×2 (RGB + mini wheel) · M4×14 ×2 (ultrasonic) · M4×25 ×4 (shield)
Cables	Quad RGB + ultrasonic each on a 10 cm mBuild cable — the guide red-crosses the 20 cm
Motors	left → EM1, right → EM2 (guide step ⑧ wiring tips)

Watch out

- The ultrasonic sensor must face exactly forward — a tilted sensor gives garbage readings for weeks.
- 10 cm cable for the sensors — teams grabbing the 20 cm one are on the step the guide crosses out in red.
- EM1/EM2 swapped = robot turns when told to go straight; check before closing up.
- Label each robot with the team name now.

◉ EXPECTED RESULT AT THE END OF THIS SESSION



redrawn from guide steps ④–⑨ (pp. 11–13)

- ☑ Robot passes all four QC points, signed by another team
- ☑ Sensors mounted straight and plugged into the correct ports
- ☑ Team label on the robot

SESSION 4 / 20 · MEET YOUR ROBOT

First Boot

Goal of this hour: Robot connects to mBlock, firmware updated, and greets the world on its screen.

YOU NEED — PIECES & MATERIALS

mBot2, assembled
fully built + QC'd
mBot2 box

Laptop + mBlock
mBlock installed
Classroom

USB cable
first pairing is wired
mBot2 box

Makeblock BT dongle
advised by Makeblock — sold separately
Sold separately

TEACH IT — THE CONCEPT

Tour the CyberPi: full-color screen, joystick, buttons A/B, speaker, microphone, light sensor, LED strip. This little brick is the robot's brain and face — everything students program flows through it.

Walk the connection ritual as a checklist students will repeat every session: power on → open mBlock → add device "CyberPi" → connect (USB first time, then Bluetooth) → update firmware if prompted. Then the classic first program: show a greeting on the screen. It is one block — and it proves the whole chain works: their code, running on their robot.

LESSON FLOW (45′ CORE)

8′	CyberPi guided tour — students point to each part as it's named.
15′	Connection ritual step by step; firmware update (this is the slow part).
12′	Program: on start → show "Hello, I am !"; add a beep and an LED color.
5′	Each robot greets the class from the front desk.
5′	Journal: write the connection ritual in your own words — it's next session's entry ticket.

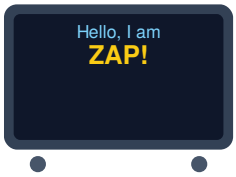
BLOCK SCRIPT (REBUILD THIS IN MBLOCK)

```
when flag clicked
  show label "Hello, I am ZAP!" on screen
  play sound  hello
  set all LEDs to  blue
```

Watch out

- Firmware updates can take minutes — do a teacher dry run before this session.
- If Bluetooth pairing fails, USB always works; don't burn class time fighting it.

◉ EXPECTED RESULT AT THE END OF THIS SESSION



every robot shows its greeting — code runs on hardware

- ☑ Robot connects and receives programs
- ☑ Greeting + sound + LED all fire on start
- ☑ Students can recite the connection ritual

SESSION 5 / 20 · MOTION & GEOMETRY

First Motion & Calibration

Goal of this hour: The robot drives exactly 50 cm — and students understand why "exactly" needs calibration.

YOU NEED — PIECES & MATERIALS

- **mBot2, assembled** — mBot2 box
- **Laptop + mBlock** — Classroom
- **Ruler / tape measure** or tape measure — Classroom
- **Masking tape** start line on floor — Classroom
- **Engineering journal** calibration table — Classroom

TEACH IT — THE CONCEPT

First motion program: drive forward 50 cm, stop. Then measure with a ruler. Robots will land at 47, 52, 49 cm… Why? Wheel slip, battery level, floor surface. This gap between **commanded** and **actual** is one of the deepest ideas in robotics — and students discover it with a ruler.

Introduce the calibration cycle: command → measure → compute error → adjust → repeat. Teams tune until three runs in a row land within ±1 cm of the 50 cm mark.

LESSON FLOW (45′ CORE)

5′	Connection ritual (entry ticket) — all robots online.
10′	Program + first run: forward 50 cm. Measure and record actual distance, three runs.
20′	Calibration cycle: adjust and re-test until 3× within ±1 cm; record the table in the journal.
10′	Class data on the board: whose robot drifted most? Same floor, same code — discuss why.

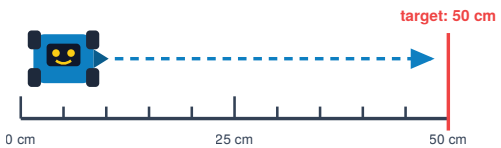
BLOCK SCRIPT (REBUILD THIS IN MBLOCK)

```
when button A pressed
  move forward 50 cm at 50 RPM
  stop moving
  show label "done" on screen
```

Watch out

- Same start line for every run — tape it on the floor.
- Low battery changes distance; note battery level next to each measurement.

◉ EXPECTED RESULT AT THE END OF THIS SESSION



measured, adjusted, repeated — robot stops on the mark

- ☑ Three consecutive runs land within ±1 cm of the mark
- ☑ Calibration table recorded (commanded / actual / error)
- ☑ Students can explain one reason commanded ≠ actual

SESSION 6 / 20 · MOTION & GEOMETRY

Shapes

Goal of this hour: The robot drives a closed square and triangle using precise turns.

YOU NEED — PIECES & MATERIALS

- **mBot2, assembled** — calibrated — mBot2 box
- **Laptop + mBlock** — Classroom
- **Masking tape** — mark the start point — Classroom
- **Ruler / tape measure** — measure the gap — Classroom

TEACH IT — THE CONCEPT

Driving a square = four times (forward, turn 90°). But which 90°? Have students walk a square themselves: at each corner they turn 90° — the **exterior** angle. For a triangle the turn is 120°, not 60°. The rule they should discover: exterior angles of any closed shape sum to 360°, so each turn = 360° ÷ number of sides.

Chalk or tape the start point. A perfect shape ends exactly where it began — that's the built-in accuracy test, and it sets up next session's competition.

LESSON FLOW (45′ CORE)

5′	Humans walk a square in the aisle; feel the exterior angle.
12′	Program the square (no loops yet — write all 8 blocks; the pain is the point).
13′	Triangle: teams predict the turn angle before testing. 60° fails beautifully.
10′	Measure gap between start and end point for the best shape; record.
5′	Journal: the 360° rule in your own words + a drawing.

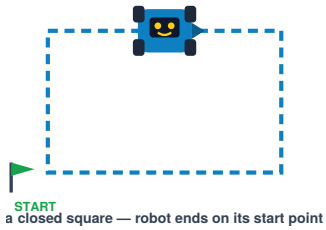
BLOCK SCRIPT (REBUILD THIS IN MBLOCK)

```
when button A pressed
  move forward 30 cm
  turn right 90°
  move forward 30 cm
  turn right 90°
  move forward 30 cm
  turn right 90°
  move forward 30 cm
  turn right 90°   <- back at start?
```

Watch out

- Let teams try 60° for the triangle and watch it spiral — the error teaches the rule.
- Turning accuracy varies with speed; slower turns are more precise.

◉ EXPECTED RESULT AT THE END OF THIS SESSION



a closed square — robot ends on its start point

- ☑ Square closes within a few cm of the start point
- ☑ Triangle drawn with the correct 120° exterior turn
- ☑ Team can state the 360° ÷ sides rule

SESSION 7 / 20 · MOTION & GEOMETRY

Pentagram Challenge

Goal of this hour: The robot draws a five-pointed star — and the team with the smallest start/end gap wins.

YOU NEED — PIECES & MATERIALS

- **mBot2, assembled** — mBot2 box
- **Laptop + mBlock** — Classroom
- **Masking tape** official start point — Classroom
- **Ruler / tape measure** gap measurement — Classroom
- **mBot2 Shield** charged for official runs — mBot2 box

TEACH IT — THE CONCEPT

A pentagram is the payoff of the 360° rule with a twist: the robot turns through 720° total (it wraps around twice), so each of the 5 turns is **144°**. Don't give the answer — give the hint "the star wraps around twice" and let teams derive it.

This is also the first competition with rules: same side length for everyone (e.g. 40 cm), same floor, three official attempts, best one counts, gap measured with a ruler. Small, fair, measurable competitions are the engine of this whole program.

LESSON FLOW (45′ CORE)

10′	Derive the turn angle in teams (hint: 720° total). Verify: 720 ÷ 5 = 144°.
20′	Build and tune the star program; calibrate turns at low speed.
12′	Official attempts: 3 runs, measure start/end gap, best counts.
3′	Podium + journal: your gap, and one tuning change that improved it.

BLOCK SCRIPT (REBUILD THIS IN MBLOCK)

```
when button A pressed
  repeat 5
    move forward 40 cm
    turn right 144°
  play sound  celebrate
```

Watch out

- Teams that discovered repeat-blocks early will use them here — let them, and showcase it as next session's topic.
- Fresh batteries for official attempts; it genuinely matters.

◉ EXPECTED RESULT AT THE END OF THIS SESSION

a five-pointed star, closed within centimeters

- ☑ Star visibly closes (gap under ~5 cm)
- ☑ Team derived 144° rather than being told
- ☑ Three official attempts recorded with measured gaps

SESSION 8 / 20 · LOOPS & EXPRESSION

Loops

Goal of this hour: Students refactor repeated code into loops and nest loops for patterns.

YOU NEED — PIECES & MATERIALS

- **mBot2, assembled** — mBot2 box
- **Laptop + mBlock** — Classroom
- **Engineering journal** — blink-count predictions — Classroom

TEACH IT — THE CONCEPT

Show the 8-block square from Session 6 next to a 3-block loop version. Same behavior, less code, one place to change. That is the whole argument for loops — and students who hand-wrote the star's five sides feel it in their fingers.

Then nest: a loop inside a loop. Example: repeat 4 [blink LEDs red–blue 3 times, then turn 90°]. Ask before each run: "how many total blinks?" Predicting the count (4 × 3 = 12) is the comprehension check.

LESSON FLOW (45′ CORE)

8′	Refactor the square into a repeat-4 loop; verify identical behavior.
10′	LED patterns with loops: police strobe, rainbow cycle, heartbeat.
17′	Nested loop challenge: square where each corner blinks 3×. Predict total blinks first.
10′	Mini-exercises: change one number to make the shape a hexagon. Which number(s)?

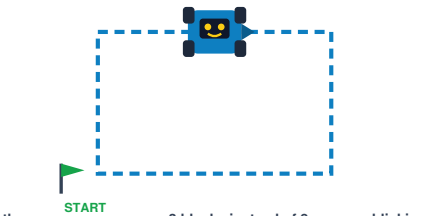
BLOCK SCRIPT (REBUILD THIS IN MBLOCK)

```
when button A pressed
  repeat 4              <- outer loop
    repeat 3            <- inner loop
      set all LEDs to red
      wait 0.2 s
      set all LEDs to blue
      wait 0.2 s
    move forward 30 cm
    turn right 90°
```

Watch out

- "How many times does the inner block run in total?" — ask it every single time.
- Hexagon answer: repeat 6 + turn 60° — two numbers change, and both come from the 360° rule.

◉ EXPECTED RESULT AT THE END OF THIS SESSION



the same square — now 3 blocks instead of 8, corners blinking

- ☑ Square rebuilt with a loop, behavior identical
- ☑ Nested-loop blink count predicted correctly before running
- ☑ Team converted the square to a hexagon by changing two numbers

SESSION 9 / 20 · LOOPS & EXPRESSION

Robot Dance

Goal of this hour: A 30-second choreography syncing motion, LEDs and sound — performed for the class.

YOU NEED — PIECES & MATERIALS

- **mBot2, assembled** — mBot2 box
- **Laptop + mBlock** — Classroom
- **Stopwatch** — 30-second limit — Classroom
- **Masking tape** — performance zone — Classroom

TEACH IT — THE CONCEPT

Creative constraint session: exactly 30 seconds, must use at least one loop, one nested pattern, motion + light + sound together. Constraints force design decisions — that's the lesson hiding inside the fun.

Timing is the technical skill here: blocks run one after another, so students must budget their 30 seconds (spins take ~1 s per 180° at medium speed, sounds have lengths). Storyboard first on paper: a timeline strip with what happens at 0 s, 5 s, 10 s…

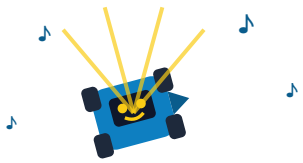
Watch out

- Cap volume — agree a max level before chaos begins.
- Clear a performance zone; spinning robots and table edges are enemies.

LESSON FLOW (45′ CORE)

5′	Brief + judging categories: most precise, most creative, best sync.
8′	Paper storyboard: the 30-second timeline.
22′	Build and rehearse; iterate on timing.
10′	Showcase: every team performs; class votes by category.

◉ EXPECTED RESULT AT THE END OF THIS SESSION



motion + LED rays + music, all in sync for 30 s

- ☑ Performance lasts 28–32 seconds
- ☑ At least one loop and one nested pattern in the code
- ☑ Motion, LEDs and sound clearly synchronized at least once

SESSION 10 / 20 · SENSING: ULTRASONIC

Echolocation

Goal of this hour: Students explain how ultrasonic sensing works and read live distances on screen.

YOU NEED — PIECES & MATERIALS

- **mBot2, assembled** — mBot2 box
- **Laptop + mBlock** — Classroom
- **Ruler / tape measure** — verify readings — Classroom
- **Boxes / obstacles** — hard + soft targets — Classroom

TEACH IT — THE CONCEPT

The ultrasonic sensor shouts (at a pitch we can't hear) and times the echo. Distance = time × speed of sound ÷ 2 — the ÷2 because sound travels there *and* back. Bats hunt with it; cars park with it. Have two students act it out: one claps, the other echoes after a delay proportional to distance.

First sensing program: show the distance on the screen forever (a loop!). Then explore: what does it read for a book at 20 cm? A soft sweater? A hand at an angle? Soft and angled surfaces scatter the echo — real sensors have real limits, and students should catalogue them.

LESSON FLOW (45′ CORE)

8′	Echolocation demo + the there-and-back ÷2 idea.
12′	Program: forever → show ultrasonic distance on screen. Ruler-check its accuracy.
15′	Sensor limits lab: hard vs. soft targets, straight vs. angled; record readings.
10′	Share the weirdest reading + explain it with the echo model.

BLOCK SCRIPT (REBUILD THIS IN MBLOCK)

```
when flag clicked
  forever
    show label (ultrasonic distance) on
screen
    wait 0.1 s
```

Watch out

- Below ~4 cm and beyond ~3 m readings go strange — let students find these edges themselves.
- The sensor sees a cone, not a laser line — nearby objects off-axis still reflect.

◉ EXPECTED RESULT AT THE END OF THIS SESSION

- ☑ Live distance updates on screen and matches a ruler within ~1 cm
- ☑ Team recorded at least two "weird" readings and explained them
- ☑ Students can explain the ÷2 in the distance formula

SESSION 11 / 20 · SENSING: ULTRASONIC

Conditionals

Goal of this hour: The robot stops by itself exactly 5 cm from a wall, from any starting distance.

YOU NEED — PIECES & MATERIALS

mBot2, assembled
mBot2 box

Laptop + mBlock
Classroom

Boxes / obstacles
the wall
Classroom

Ruler / tape measure
stopping-gap check
Classroom

TEACH IT — THE CONCEPT

Enter **if**: for the first time, the robot makes a decision. “If distance < 10 cm → stop” inside a forever loop is the sense–decide–act loop made real — point back to the Session 1 poster; this is that diagram, running.

Then the precision challenge: stop at exactly 5 cm. Naive code overshoots — the robot is fast and checks the sensor only every loop pass. Students discover the fix themselves: approach slower, or slow down as the wall gets close (two thresholds: far → fast, near → slow, at 5 → stop). That layered answer previews proportional control in Grade 9.

LESSON FLOW (45′ CORE)

8′	Build stop-if-close v1; watch it overshoot at full speed.
20′	Iterate: slower approach, then two-speed approach. Measure the stopping gap each attempt.
12′	Official challenge: 3 runs from different starting distances; average gap recorded.
5′	Journal: why does speed change the stopping distance?

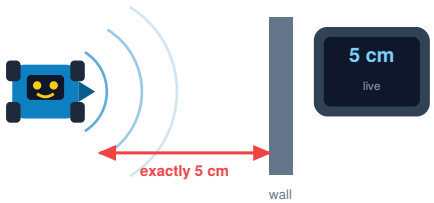
BLOCK SCRIPT (REBUILD THIS IN MBLOCK)

```
when button A pressed
  forever
    if ultrasonic distance > 20
      move forward at 50 RPM
    else if ultrasonic distance > 5
      move forward at 15 RPM   <- creep
    else
      stop moving
      play sound  beep
```

Watch out

- A robot that never stops usually has the if outside the forever loop — the #1 bug today.
- Tell teams the two-speed trick exists only if they’re stuck after 10 minutes.

◉ EXPECTED RESULT AT THE END OF THIS SESSION



- ☑ Robot stops without human help from any starting distance
- ☑ Average stopping gap within ~1 cm of the 5 cm target
- ☑ Code uses if/else inside a forever loop

SESSION 12 / 20 · SENSING: ULTRASONIC

Wander Mode

Goal of this hour: The robot roams the arena indefinitely, avoiding every obstacle on its own.

YOU NEED — PIECES & MATERIALS

- **mBot2, assembled** — mBot2 box
- **Laptop + mBlock** — Classroom
- **Boxes / obstacles** — arena walls + obstacles — Classroom
- **Stopwatch** — longest-run timing — Classroom

TEACH IT — THE CONCEPT

Combine everything: drive until something is close, then back up, turn, continue — forever. This is a real autonomous behavior, the same core logic as a robot vacuum.

Add randomness: instead of always turning 90°, turn a random angle between 60° and 120°. Ask why that helps — a fixed angle can trap the robot bouncing in a corner forever; randomness breaks the pattern. (This is a genuinely deep idea: randomness as an escape from loops.)

LESSON FLOW (45′ CORE)

10′	Build wander v1 with a fixed 90° turn; test in the arena.
10′	Find the corner trap (or stage one); introduce the random block.
15′	Tune: detection threshold, backup distance, random range. Longest run wins.
10′	Arena timing: longest continuous run without touching a wall.

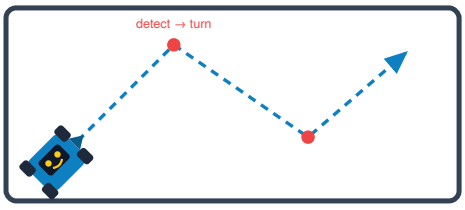
BLOCK SCRIPT (REBUILD THIS IN MBLOCK)

```
when button A pressed
  forever
    if ultrasonic distance < 12
      move backward 10 cm
      turn right (random 60 to 120)°
    else
      move forward at 40 RPM
```

Watch out

- Threshold too small → robot kisses walls; too big → robot is scared of everything. Tuning IS the lesson.
- Dark, soft obstacles reflect badly — use boxes, not backpacks.

◉ EXPECTED RESULT AT THE END OF THIS SESSION



robot roams the arena — never touches a wall

- ☑ Robot runs 60+ seconds in the arena with no wall contact
- ☑ Random turn range used and justified
- ☑ Team tuned at least two parameters and logged the effect

SESSION 13 / 20 · SENSING: ULTRASONIC

Arena + Debug Clinic ✱

Goal of this hour: Teams push wander mode to a personal best and practice structured debugging.

YOU NEED — PIECES & MATERIALS

mBot2, assembled
mBot2 box

Laptop + mBlock
Classroom

Boxes / obstacles
arena
Classroom

Stopwatch
Classroom

Engineering journal
bug report — graded
Classroom

TEACH IT — THE CONCEPT

Today formalizes debugging as a four-step discipline: **predict** what the code should do → **test** → **observe** exactly what happened (not “it’s broken”, but “it turns left when the wall is on the left”) → **fix one thing** and re-test. One change at a time — change three things and you learn nothing.

Every team must produce one written bug report today: symptom, hypothesis, fix, result. Debugging is the most transferable skill in this whole course; treat the bug report as a first-class deliverable, not paperwork.

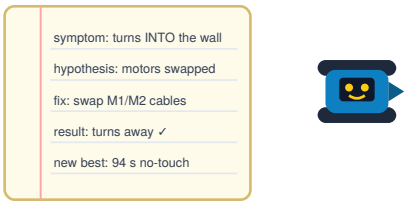
Watch out

- Buffer session: if the schedule slipped, this is the hour that absorbs it.
- Ban the sentence “it doesn’t work” — require a symptom sentence instead.

LESSON FLOW (45′ CORE)

5′	Model one live debug on a teacher robot using the four steps aloud.
25′	Teams improve their wander bots; every change logged as predict/test/observe/fix.
10′	Official arena runs — beat your Session 12 time.
5′	Teams read their best bug report aloud; class picks the “bug of the day”.

◉ EXPECTED RESULT AT THE END OF THIS SESSION



a real bug report — the deliverable of the day

- ☑ New personal best arena time recorded
- ☑ At least one complete four-step bug report in the journal
- ☑ No change was made without a prediction first

SESSION 14 / 20 · LINE FOLLOWING

Line Following I

Goal of this hour: The robot follows the printed line using simple left/right correction.

YOU NEED — PIECES & MATERIALS

- **mBot2, assembled** — mBot2 box
- **Laptop + mBlock** — Classroom
- **Line-following map** — in the mBot2 box — mBot2 box
- **Masking tape** — tape it flat! — Classroom
- **Phone (video/ angle)** — slow-mo the wobble — Classroom

TEACH IT — THE CONCEPT

The quad-RGB sensor under the nose reads how dark the floor is at four points. On the included map: white floor, black line. The simplest follower (“bang-bang”): if the line drifts left under the sensor → steer left; if right → steer right. Two rules, and the robot follows a curve. Magic — briefly.

First: the calibration ritual. Every floor and lighting is different, so the sensor must learn “white here” vs. “black here” (hold the calibrate button, sweep across the line). Make it a ritual like the connection checklist — uncalibrated sensors cause 80% of line-following tears.

Let students watch closely: the robot zig-zags. Name it — **the wobble** — and write it on the board as an open question. It gets answered in Grade 9, and planting it now is the point.

LESSON FLOW (45′ CORE)

8′	Sensor calibration ritual on the map — every team, every robot.
15′	Build bang-bang follower; first laps on the oval.
12′	Observe and sketch the wobble in the journal; slow-motion phone video helps.
10′	Wobble question on the board: why does it zig-zag? Collect hypotheses — don’t answer.

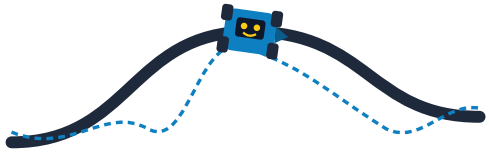
BLOCK SCRIPT (REBUILD THIS IN MBLOCK)

```
when button A pressed
  forever
    if line sensor detects line on LEFT
side
      turn left slightly (L 20 / R 50 RPM)
    else if line detected on RIGHT side
      turn right slightly (L 50 / R 20 RPM)
    else
      move forward at 45 RPM
```

Watch out

- Re-calibrate whenever the robot moves to a different floor or the sun moves — seriously.
- Map curls at the edges cause false readings; tape it flat.

◉ EXPECTED RESULT AT THE END OF THIS SESSION



robot follows the line — notice the zig-zag “wobble”

- ☑ Robot completes a full lap of the oval unassisted
- ☑ Calibration ritual performed from memory
- ☑ Wobble observed, sketched, and hypothesized about

SESSION 15 / 20 · LINE FOLLOWING

Line Following II

Goal of this hour: A smoother, faster follower using three states — and tuned speed values.

YOU NEED — PIECES & MATERIALS

- **mBot2, assembled** — mBot2 box
- **Laptop + mBlock** — Classroom
- **Line-following map** — calibrated on THIS floor — mBot2 box
- **Stopwatch** — lap times — Classroom
- **Engineering journal** — tuning log — Classroom

TEACH IT — THE CONCEPT

Upgrade from two rules to three states: line under the LEFT sensors → steer left; CENTER → drive straight; RIGHT → steer right. The new “straight” state is what reduces the wobble: the robot stops over-correcting when it’s already fine.

Then tuning: base speed vs. correction strength. Fast base speed + weak correction = flies off curves. Slow + strong = crawls but never loses the line. Teams find their own sweet spot through timed laps — there is no single right answer, only trade-offs. Write “trade-off” on the board; it’s the engineering word of the day.

LESSON FLOW (45′ CORE)

10′	Add the center/straight state; verify reduced wobble.
20′	Tuning sprints: change ONE value, run a timed lap, log it. Repeat.
10′	Best-lap trials: three official timed laps, best counts.
5′	Journal: your final values + the trade-off you chose.

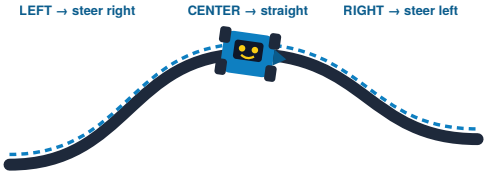
BLOCK SCRIPT (REBUILD THIS IN MBLOCK)

```
forever
  if line under CENTER sensors
    move forward at 55 RPM        <-
straight state
  else if line under LEFT
    set motors  L 15 / R 55 RPM
  else if line under RIGHT
    set motors  L 55 / R 15 RPM
```

Watch out

- One value per change — the debug discipline from Session 13 applies to tuning too.
- Losing the line at the same curve every lap = that curve is tighter than your correction strength.

◉ EXPECTED RESULT AT THE END OF THIS SESSION



three states, visibly smoother trajectory

- ☑ Lap time improved vs. Session 14
- ☑ Robot recovers the line on the tightest curve of the map
- ☑ Tuning log shows one-change-at-a-time discipline

SESSION 16 / 20 · LINE FOLLOWING

Events ✱

Goal of this hour: The robot starts on a button press and reacts to color patches on the track.

YOU NEED — PIECES & MATERIALS

- **mBot2, assembled** — mBot2 box
- **Laptop + mBlock** — Classroom
- **Line-following map** — mBot2 box
- **Color patches** — matte paper, red + green — Classroom
- **Stopwatch** — Classroom

TEACH IT — THE CONCEPT

Events flip the program's shape: instead of one long script, code now waits for things to happen — a button press, a color under the sensor. "When button B pressed → start following" turns the robot into something you operate, not just launch.

The quad-RGB sensor also reads **color**, not just dark/light. Add traffic rules to the track: a red patch = stop 2 s, a green patch = beep and continue. Students place the patches themselves — designing the track is half the fun and all of the ownership. This session is the direct rehearsal for the capstone's rescue-zone stop.

LESSON FLOW (45′ CORE)

8′	Restructure: line following starts on button B, stops on button A.
18′	Add color rules: red patch → pause 2 s; green → beep + continue. Place patches on the track.
14′	Best-lap trials on the enriched track — rules must be obeyed to count.
5′	Journal: list every event your program now responds to.

BLOCK SCRIPT (REBUILD THIS IN MBLOCK)

```
when button B pressed
  set  running  to 1

when button A pressed
  set  running  to 0
  stop moving

forever
  if running = 1
    ...line following...
    if color sensor sees RED
      stop moving, wait 2 s
    if color sensor sees GREEN
      play sound  beep
```

Watch out

- Buffer session — compress here if behind; the color rules can shrink to red-only.
- Glossy tape reflects oddly; matte paper patches read more reliably.

◉ EXPECTED RESULT AT THE END OF THIS SESSION

the robot reacts to buttons and color patches on the track

- ☑ Robot starts/stops on buttons only — no re-flashing between runs
- ☑ Both color rules obeyed on a full lap
- ☑ Team designed and placed its own patch layout

SESSION 17 / 20 · CAPSTONE: RESCUE RUN

Capstone: Plan

Goal of this hour: A signed-off paper plan: flowchart of the full Rescue Run mission.

YOU NEED — PIECES & MATERIALS

Engineering journal
flowchart — today's deliverable
Classroom

Printed cards
rubric handout
Classroom

Line-following map
course planning reference
mBot2 box

mBot2, assembled
reference only — no coding today
mBot2 box

TEACH IT — THE CONCEPT

Reveal the mission: follow the line through the course, handle a surprise obstacle ON the line, stop precisely inside the rescue zone (a colored patch), celebrate with lights + sound. Everything needed has been built in Sessions 10–16 — today is about **assembling** known pieces, which is what real engineering mostly is.

No code today. Teams produce a flowchart: boxes for states (following / avoiding / stopped / celebrating), arrows for the events that switch between them. The teacher sign-off gate is strict: a plan that can't answer "what does the robot do when X?" goes back for revision. Planning discipline now saves two sessions of chaos later.

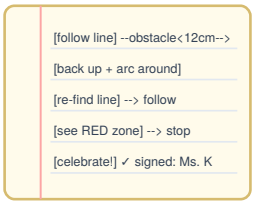
Watch out

- Refuse vague boxes like "do the thing" — every box must be a behavior they have already built.
- The obstacle sits ON the line: teams must realize avoidance means leaving and re-finding the line.

LESSON FLOW (45′ CORE)

8′	Mission briefing + rubric walkthrough (autonomy, precision, code quality, presentation).
10′	Decompose in teams: list the sub-behaviors and which past session built each.
20′	Draw the flowchart; walk it with a finger while a teammate plays "robot".
7′	Sign-off interviews: teacher asks two "what if…" questions per team.

◉ EXPECTED RESULT AT THE END OF THIS SESSION



a signed flowchart — the ticket to touch the keyboard

- ☑ Flowchart covers all four mission phases
- ☑ Team answered both "what if" questions without hand-waving
- ☑ Teacher signature on the plan

SESSION 18 / 20 · CAPSTONE: RESCUE RUN

Capstone: Build I

Goal of this hour: Line following + precise rescue-zone stop working end-to-end.

YOU NEED — PIECES & MATERIALS

- **mBot2, assembled** — mBot2 box
- **Laptop + mBlock** — Classroom
- **Line-following map** — capstone course — mBot2 box
- **Color patches** — rescue zone — Classroom
- **Masking tape** — course layout — Classroom

TEACH IT — THE CONCEPT

Build in the order of the flowchart, and integrate one piece at a time: first, clean line following on the capstone course; then the red-zone stop. Resist teams that want to build everything at once — “make one arrow of the flowchart work, then the next” is the mantra.

The zone stop reuses Session 16’s color event, but precision matters now: the robot must halt *inside* the zone. If it slides past, lower approach speed near the zone — the Session 11 two-speed idea returns, and students should recognize it.

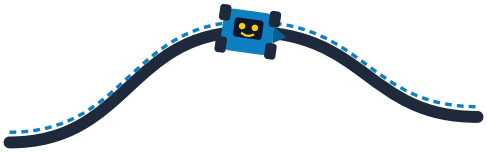
Watch out

- Keep the obstacle OFF the course today — one variable at a time.
- Celebration code is morale fuel; let them make it loud (within the volume pact).

LESSON FLOW (45′ CORE)

5′	Flowchart on the table; agree today’s target: phases 1 + 4 (follow, stop, celebrate).
30′	Build + test line → zone stop → celebration on the real course.
10′	Milestone demo to the teacher: three clean follow-and-stop runs.

◉ EXPECTED RESULT AT THE END OF THIS SESSION



follow → stop inside the zone → celebrate, three times in a row

- ☑ Three consecutive runs: full lap + stop inside the zone
- ☑ Celebration triggers only in the zone, not on other colors
- ☑ Code matches the flowchart phases 1 and 4

SESSION 19 / 20 · CAPSTONE: RESCUE RUN

Capstone: Build II ✱

Goal of this hour: The full mission runs: obstacle handled, line re-found, zone stop preserved.

YOU NEED — PIECES & MATERIALS

mBot2, assembled
mBot2 box

Laptop + mBlock
Classroom

Line-following map
mBot2 box

Color patches
Classroom

Boxes / obstacles
the obstacle, ON the line
Classroom

mBot2 Shield
note battery level in journal
mBot2 box

TEACH IT — THE CONCEPT

Integrate the hard part: the obstacle sits on the line, so the robot must notice it (ultrasonic, Session 11), leave the line, arc around (motion, Sessions 5–7), and re-acquire the line (a search behavior — the one genuinely new piece). Simplest re-find: after the arc, drive forward slowly until any line sensor fires, then resume following.

Then rehearse under competition conditions: full runs, timed, from the official start. Teams should also decide their run-day parameters (speeds, thresholds) and freeze them — tinkering minutes before an official run is how good robots die.

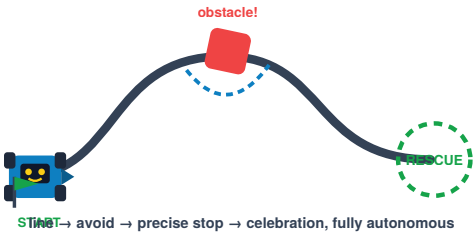
Watch out

- Buffer session — a struggling class can make the obstacle position fixed and known.
- The arc must be wide enough to clear the obstacle but tight enough to land near the line — pure tuning.

LESSON FLOW (45′ CORE)

25′	Build the avoid-and-refind behavior; test with the obstacle at different points on the line.
13′	Full dress rehearsals from the official start line; log completion of each phase.
7′	Freeze parameters; write them in the journal as the “race configuration”.

◉ EXPECTED RESULT AT THE END OF THIS SESSION



- ☑ At least one full clean run: follow → avoid → re-find → zone stop → celebrate
- ☑ Re-find works with the obstacle at two different positions
- ☑ Race configuration frozen and written down

SESSION 20 / 20 · CAPSTONE: RESCUE RUN

Scored Runs & Reflection

Goal of this hour: Two official scored attempts, a short team presentation, and an honest retrospective.

YOU NEED — PIECES & MATERIALS

- **mBot2, assembled** — race configuration frozen — mBot2 box
- **Laptop + mBlock** — Classroom
- **Line-following map** — mBot2 box
- **Color patches** — Classroom
- **Boxes / obstacles** — Classroom
- **Stopwatch** — + visible scoreboard — Classroom
- **mBot2 Shield** — charged — official runs — mBot2 box

TEACH IT — THE CONCEPT

Run it like an event: official start line, a visible scoreboard (phases completed × points, small time bonus), two attempts per team, best counts. Between attempts teams may fix code — under time pressure, exactly like a real robotics competition pit.

The presentation matters as much as the run: 2 minutes per team — what worked, what failed, what you'd do next. Normalize talking about failure; the team whose robot crashed but who can explain *why* deserves loud applause. Close by seeding next year: "your robot gets a face and tracks."

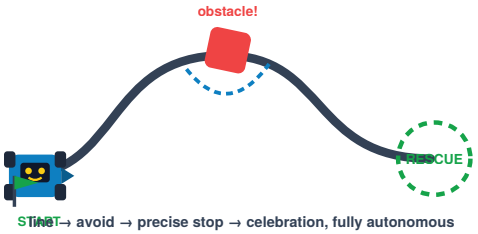
Watch out

- Score phases, not just full success — a robot that avoids but misses the zone still earns points.
- Photograph the scoreboard; it opens Grade 8, Session 1's review quiz.

LESSON FLOW (45′ CORE)

8′	Setup, scoreboard, attempt order drawn by lot.
22′	Official attempts (2 per team) with pit time between rounds.
10′	Team presentations: worked / failed / next — 2 minutes each.
5′	Individual reflection in the journal + Grade 8 teaser.

◉ EXPECTED RESULT AT THE END OF THIS SESSION



- ☑ Every team completed two official attempts
- ☑ Scores recorded phase by phase on the public board
- ☑ Every student wrote a personal what-I'd-do-next reflection